DEFINITION You *register* services with a DI container so that it knows which implementation to use for each requested service. This registration typically takes the form “For interface X, use implementation Y.”

Exactly how you register your interfaces and types with a DI container can vary depending on the specific DI container implementation, but the principles are generally the same. ASP.NET Core includes a simple DI container out of the box, so let’s look at how it’s used during a typical request.

8.3 Using dependency injection in ASP.NET Core

ASP.NET Core was designed from the outset to be modular and composable, with an almost plugin-style architecture, which is generally complemented by DI. Consequently, ASP.NET Core includes a simple DI container that all the framework libraries use to register themselves and their dependencies.

This container is used, for example, to register the minimal API infrastructure—the formatters, the Kestrel web server, and so on. It’s a basic container, so it exposes only a few methods for registering services, but you have the option to replace it with a third-party DI container that gives you extra capabilities, such as autoregistration and setter injection. The DI container is built into the ASP.NET Core hosting model, as shown in figure 8.4.